



Module 13 — Git & GitHub pour Data Analyst

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 13



Ce que tu seras capable de faire à la fin de ce module

- Expliquer pourquoi le versioning change la vie d'un-e Data Analyst
- Utiliser les commandes Git de base : `init`, `add`, `commit`, `log`, `status`, `clone`
- Écrire un `.gitignore` adapté à un projet data
- Créer un dépôt GitHub, `push`/`pull`, rédiger un README, ouvrir une Pull Request
- Te repérer sur GitLab et Azure Repos si ton entreprise n'utilise pas GitHub



Durée estimée : 75 minutes  **Outils** : Git (ligne de commande) · GitHub · notions GitLab/Azure Repos  **Prérequis** : Niveau Débutant terminé

1. Pourquoi le versioning pour un Data Analyst

Le scénario que tu as déjà vécu

Tu travailles sur un rapport pour ta direction. Tu envoies `rapport_ventes.xlsx` par mail à un collègue pour relecture. Il te renvoie `rapport_ventes_v2.xlsx` avec ses commentaires. Toi, pendant ce temps, tu as continué à travailler sur ta version. Le lendemain, tu as

`rapport_ventes_v2_FINAL.xlsx`, `rapport_ventes_v2_FINAL_vraiment.xlsx`, et plus personne ne sait quelle version contient les bons chiffres.

C'est exactement le problème que **Git** résout — pas seulement pour du code, mais pour tout fichier texte : scripts Python/R, requêtes SQL, notebooks, fichiers de configuration.

Ce que Git apporte concrètement

SANS GIT	AVEC GIT
<code>analyse_final_v3.py</code>	Un seul fichier <code>analyse.py</code> , un historique complet de ses versions
"Qui a supprimé cette ligne de code la semaine dernière ?"	<code>git log</code> te montre qui, quand, et pourquoi (message de commit)
Deux personnes modifient le même script en même temps → l'une écrase le travail de l'autre	Git détecte le conflit et te force à le résoudre proprement
Ton script casse et tu ne sais pas revenir en arrière	<code>git checkout</code> restaure n'importe quelle version passée
Ton travail n'existe que sur ton ordinateur	Un dépôt distant (GitHub) sauvegarde et partage ton travail

💡 **Pour un DA**, Git sert surtout à : versionner tes scripts d'analyse (Python/R/SQL), collaborer sur un projet avec d'autres analystes, et **publier ton travail** (un dashboard, un rapport, une app) — ce qu'on fera au Mini-Projet C et au Projet Final de ce niveau.

2. Concepts Git fondamentaux

Git est un système de **contrôle de version décentralisé**. Quatre notions à maîtriser avant de taper la moindre commande :

CONCEPT	DÉFINITION
Repository (dépôt)	Le dossier de ton projet, suivi par Git — contient tout l'historique
Commit	Une "photo" de l'état de tes fichiers à un instant donné, avec un message qui explique le changement
Staging area (index)	La zone d'attente où tu places les fichiers que tu veux inclure dans ton prochain commit
Remote (dépôt distant)	Une copie de ton dépôt hébergée ailleurs (GitHub, GitLab, Azure Repos...) — pour sauvegarder et collaborer

Le cycle de vie d'un fichier

Dossier de travail → Staging area → Dépôt local → Dépôt distant
 (git add) (git commit) (git push)

Tu modifies un fichier → tu le mets en scène avec `git add` → tu l'enregistres dans l'historique avec `git commit` → tu l'envoies vers GitHub avec `git push`.

 **Pont Excel → Git :** `git commit` est l'équivalent d'un "Enregistrer sous" horodaté et commenté — sauf que Git garde TOUTES les versions, pas seulement la dernière, et te permet de comparer ou de revenir à n'importe laquelle.

3. Git en pratique — les commandes de base

Installer et configurer Git

```
BASH
git --version
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@exemple.com"
```

Initialiser un dépôt

```
BASH
mkdir mon-projet-analyse
cd mon-projet-analyse
git init
```

Le cycle add → commit → log

```
BASH
# Voir l'état de ton dossier (fichiers modifiés, nouveaux, etc.)
git status

# Ajouter un fichier à la staging area
git add analyse_ventes.py

# Ajouter TOUS les fichiers modifiés
git add .

# Enregistrer un commit avec un message clair
git commit -m "Ajoute le script de nettoyage des ventes régionales"

# Consulter l'historique
git log
git log --oneline    # version compacte, un commit par ligne
```

⚠ **Un bon message de commit** décrit le *pourquoi*, pas juste le *quoi*. `"fix"` ne dit rien. `"Corrige le filtre qui excluait les ventes à 0 FCFA"` raconte une histoire utile dans 6 mois.

Le fichier `.gitignore`

Certains fichiers ne doivent **jamais** être versionnés : données volumineuses, mots de passe, fichiers temporaires générés automatiquement.

```
# .gitignore – projet data typique
*.csv
*.xlsx
.env
__pycache__/
.ipynb_checkpoints/
venv/
```

💡 Pourquoi exclure les `.csv` / `.xlsx` ? Parce que les données brutes sont souvent volumineuses, parfois confidentielles, et ne devraient pas transiter par un historique Git public. On versionne le **code qui traite la donnée**, rarement la donnée elle-même.

Récupérer un dépôt existant

```
BASH
git clone https://github.com/nom-utilisateur/nom-du-depot.git
```

4. GitHub en pratique

GitHub héberge ton dépôt Git "dans le cloud" et ajoute des fonctionnalités de collaboration (Pull Requests, issues, README, pages web...).

Créer et connecter un dépôt

1. Sur github.com, clique `New repository`, donne-lui un nom (ex: `audit-sante-ci`), coche "Add a README file".
2. Relie ton dépôt local à GitHub :

BASH

```
git remote add origin https://github.com/nom-utilisateur/audit-sante-ci.git
git branch -M main
git push -u origin main
```

3. Pour récupérer les changements faits par quelqu'un d'autre (ou depuis un autre ordinateur) :

BASH

```
git pull origin main
```

Le README — ta vitrine

Chaque dépôt GitHub sérieux commence par un `README.md` qui explique : à quoi sert le projet, comment l'exécuter, quelles données il utilise. C'est souvent la première chose qu'un recruteur regarde sur ton profil GitHub.

La Pull Request (PR)

Quand tu travailles à plusieurs (ou que tu veux documenter un changement avant de l'intégrer), tu crées une **branche**, tu y pousSES ton travail, puis tu ouvres une **Pull Request** : une proposition de fusionner tes changements dans la branche principale, avec un espace de discussion et de relecture.

BASH

```
git checkout -b ajout-analyse-regionale
# ... tu travailles, tu commit ...
git push -u origin ajout-analyse-regionale
# Puis sur GitHub : "Compare & pull request"
```

💡 On utilisera exactement ce mécanisme (dépôt GitHub → déploiement) pour publier le Mini-Projet C et le Projet Final de ce niveau sur Streamlit Community Cloud.

5. Pont GitHub → GitLab → Azure Repos

GitHub n'est pas le seul endroit où tu croiseras Git en entreprise. Selon l'écosystème technique de ton employeur, tu tomberas sur **GitLab** (fréquent en Europe, souvent auto-hébergé pour des raisons de confidentialité) ou **Azure Repos** (quasi systématique dans les entreprises déjà équipées Microsoft — cohérent avec le Power BI que tu utiliseras plus loin dans ce niveau).

Bonne nouvelle : **Git reste Git**. Les commandes `add` / `commit` / `push` / `pull` sont identiques partout. Seuls le vocabulaire et l'interface web changent.

CONCEPT	GITHUB	GITLAB	AZURE REPOS
Dépôt	Repository	Project (contient un dépôt)	Repository
Proposition de fusion	Pull Request	Merge Request	Pull Request
Intégration continue	GitHub Actions	GitLab CI/CD	Azure Pipelines
Suivi de tâches	Issues	Issues	Work Items (Azure Boards)
Hébergement typique	Cloud (github.com)	Cloud ou auto-hébergé	Cloud (Azure DevOps), écosystème Microsoft
Où tu le croises	Startups, open source, portfolio perso	Entreprises soucieuses de souveraineté des données, équipes DevOps	Grandes entreprises déjà sous Microsoft 365 / Azure

 **Pont pratique** : une "Merge Request" GitLab et une "Pull Request" GitHub/Azure, c'est le même concept sous un nom différent — ne te laisse pas dérouter le jour où tu changes d'employeur.

Pour ce bootcamp, on utilise GitHub (le plus répandu, gratuit, et directement compatible avec Streamlit Community Cloud) — mais tout ce que tu apprends ici (les commandes Git, le concept de PR/MR) se transpose directement.

✓ Résumé du module

CONCEPT	CE QU'IL FAUT RETENIR
Pourquoi Git	Historique, collaboration sans écraser le travail des autres, retour arrière possible
Repository	Dossier de projet suivi par Git
Commit	Photo horodatée et commentée de l'état des fichiers
<code>git add</code> / <code>git commit</code>	Mettre en scène puis enregistrer un changement
<code>git status</code> / <code>git log</code>	Voir l'état actuel / l'historique
<code>.gitignore</code>	Exclure données volumineuses, secrets, fichiers temporaires
<code>git clone</code> / <code>git push</code> / <code>git pull</code>	Récupérer / envoyer / synchroniser avec un dépôt distant
Pull Request (PR)	Proposer une fusion de changements, avec relecture
GitLab / Azure Repos	Mêmes concepts Git, vocabulaire différent (Merge Request = Pull Request)

🧠 Quiz — Vérifie ta compréhension

Réponds mentalement, puis clique sur **"Voir la réponse"** pour te corriger.

Q1. Tu viens de modifier `analyse.py`. Quelle est la bonne séquence pour l'enregistrer dans l'historique Git ?

- a) `git commit -m "message"` puis `git add analyse.py`
- b) `git add analyse.py` puis `git commit -m "message"`
- c) `git push` puis `git add analyse.py`

👉 Voir la réponse

✓ b) — Il faut d'abord mettre le fichier en staging area avec `git add`, puis créer le commit avec `git commit -m "message"`. `git push` n'intervient qu'après, pour envoyer vers un dépôt distant.

Q2. Pourquoi met-on généralement les fichiers `.csv` et `.xlsx` dans `.gitignore` sur un projet data ?

- a) Git ne sait pas lire les fichiers Excel
- b) Ils sont souvent volumineux, parfois confidentiels, et ce n'est pas le rôle de Git de versionner la donnée elle-même
- c) `.gitignore` est obligatoire pour que `git init` fonctionne

👉 Voir la réponse

✓ b) — On versionne le code qui traite la donnée, pas la donnée brute. Elle est souvent trop volumineuse pour un historique Git, et peut contenir des informations sensibles qu'on ne veut pas exposer (surtout sur un dépôt public).

Q3. Ton collègue utilise GitLab, toi GitHub. Il te parle d'une "Merge Request" qu'il vient d'ouvrir. De quoi parle-t-il ?

- a) D'un concept propre à GitLab, sans équivalent sur GitHub
- b) De l'équivalent GitLab d'une Pull Request GitHub — proposer de fusionner des changements, avec relecture
- c) D'une commande pour fusionner deux dépôts Git différents

👉 Voir la réponse

✓ b) — "Merge Request" (GitLab) et "Pull Request" (GitHub, Azure Repos) désignent exactement le même concept : une proposition de fusionner une branche dans une autre, avec un espace de discussion et de relecture avant validation.

Q4. Tu rejoins une entreprise 100% équipée Microsoft 365. Quelle plateforme Git as-tu le plus de chances d'y trouver ?

- a) GitHub uniquement
- b) GitLab auto-hébergé
- c) Azure Repos

 [Voir la réponse](#)

✓ **c)** — Azure Repos s'intègre nativement à l'écosystème Microsoft (Azure DevOps, Microsoft 365, souvent aux côtés de Power BI) — c'est le choix le plus fréquent dans les entreprises déjà installées sur cette stack.

Module suivant

Tu sais maintenant versionner et partager ton travail. Dans le **Module 14**, on passe au nettoyage de données robuste — expressions régulières, debugging, documentation.

→ **Module 14 — Data Cleaning robuste**